

CSE 3200: System Development Project

# **MEDFLOW: AN INTEGRATED OUTPATIENT CARE COORDINATION SYSTEM**

By

**Apu Sharma**

Roll: 2107005

**&**

**Md. Eftakhar Jaman Arfan**

Roll: 2107030



**Department of Computer Science and Engineering**

**Khulna University of Engineering & Technology**

**Khulna 9203, Bangladesh**

**April, 2026**

# **MedFlow: An Integrated Outpatient Care Coordination System**

By

**Apu Sharma**

Roll: 2107005

**&**

**Md. Eftakar Jaman Arfan**

Roll: 2107030

**Supervisor:**

**Dr. K. M. Azharul Hasan**

Professor

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

---

Signature

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Khulna 9203, Bangladesh

**April, 2026**

## **Acknowledgment**

All the praise to Almighty Allah, whose blessings and mercy enabled us to complete this project successfully. We express our profound gratitude to our supervisor, Dr. K. M. Azharul Hasan, Professor, Department of Computer Science and Engineering, Khulna University of Engineering & Technology, for his invaluable guidance, constant encouragement, and constructive feedback throughout the course of this work.

We also extend our sincere thanks to the faculty members of the Department of Computer Science and Engineering at KUET for their continuous support and for equipping us with the academic foundation required to undertake a project of this scope. We are grateful to the open-source communities behind NestJS, Prisma, React, and Flutter, whose documentation and tooling made this work possible. Finally, we thank our families and friends for their unwavering moral support throughout the entire duration of this project.

**Authors**

## Abstract

The present healthcare delivery system in Bangladesh comprises of an inefficient booking of appointments, fragmented clinical data, and ineffective coordination of physicians with pharmaceutical suppliers and diagnostic centres, and real-time communication among care providers; this project addresses this gap with **MedFlow: An Integrated Outpatient Care Coordination System** deployed as a NestJS-Prisma-PostgreSQL backend, a React web dashboard, and This system has 5 roles (Patient, Doctor, Pharmacy, Diagnostic, and Admin), role-scoped JWT/RBAC security and outpatient operations are modelled by seven state appointment workflow, five state prescription workflow, and four state lab-order workflow. Web Socket is included in it as well. Real-time notifications at the IO level, processing of prescription and diagnostic documents by Cloudinary, managing profiles, password-reset features, and audit logs in a manner that any notable clinical activity can be tracked and reported to the necessary user. Doctors, pharmacies and diagnostic centres can use role-specific dashboards to organize appointments, prescriptions, reports, notifications and profile data on the web platform and use the Flutter mobile application to cover the patient-facing workflow of doctor browsing, appointment-booking, accessing medical-records, viewing prescription, downloading reports and of receiving notifications. The system had a ten week iterative process of staged analysis, design, implementation and testing process and had automated tests in the backend, web and mobile codebase and had been tested using scenario-driven functional validation of the core clinical workflows. The prototype resulting from it suggests that a multi-platform, software design with type safety and modular features could be a valid and scalable framework to facilitate integrated outpatient care coordination and reduce administrative overhead, improve status visibility, and increase the communication between patients and service providers.

# Contents

|                                                               |             |
|---------------------------------------------------------------|-------------|
| <b>Title Page</b>                                             | <b>i</b>    |
| <b>Acknowledgment</b>                                         | <b>ii</b>   |
| <b>Abstract</b>                                               | <b>iii</b>  |
| <b>Contents</b>                                               | <b>iv</b>   |
| <b>List of Tables</b>                                         | <b>vii</b>  |
| <b>List of Figures</b>                                        | <b>viii</b> |
| <b>I Introduction</b>                                         | <b>1</b>    |
| 1.1 Introduction . . . . .                                    | 1           |
| 1.2 Background / Problem Statement . . . . .                  | 1           |
| 1.3 Objectives . . . . .                                      | 2           |
| 1.4 Scope . . . . .                                           | 2           |
| 1.5 Unfamiliarity of the Problem / Topic / Solution . . . . . | 2           |
| 1.6 Project Planning and Work Distribution . . . . .          | 3           |
| 1.7 Applications of the Work . . . . .                        | 3           |
| 1.8 Organization of the Project . . . . .                     | 4           |
| <b>II Related Works</b>                                       | <b>5</b>    |
| 2.1 Introduction . . . . .                                    | 5           |
| 2.2 Related Works . . . . .                                   | 5           |
| 2.2.1 Electronic Health Record Systems . . . . .              | 5           |
| 2.2.2 Commercial Appointment APIs . . . . .                   | 5           |
| 2.2.3 Pharmacy and Laboratory Information Systems . . . . .   | 6           |
| 2.2.4 Clinical Workflow Backends . . . . .                    | 6           |
| 2.2.5 Flutter Healthcare Applications . . . . .               | 6           |
| 2.3 Discussion . . . . .                                      | 6           |

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>III System Analysis and Design</b>                                              | <b>8</b>  |
| 3.1 Introduction . . . . .                                                         | 8         |
| 3.2 System Analysis and Design Approach . . . . .                                  | 8         |
| 3.3 Overall Framework and Architecture . . . . .                                   | 8         |
| 3.3.1 Data Flow Diagram . . . . .                                                  | 10        |
| 3.3.2 Backend Module Structure . . . . .                                           | 10        |
| 3.3.3 Web Frontend Architecture . . . . .                                          | 10        |
| 3.3.4 Mobile Frontend Architecture . . . . .                                       | 10        |
| 3.4 Problem Design and Analysis . . . . .                                          | 12        |
| 3.4.1 Role Identification . . . . .                                                | 12        |
| 3.4.2 Appointment State Machine . . . . .                                          | 12        |
| 3.4.3 Prescription State Machine . . . . .                                         | 18        |
| 3.4.4 Lab Order State Machine . . . . .                                            | 18        |
| 3.5 Database Schema Design . . . . .                                               | 18        |
| 3.6 Conclusion . . . . .                                                           | 23        |
| <b>IV Implementation, Results and Discussions</b>                                  | <b>24</b> |
| 4.1 Introduction . . . . .                                                         | 24        |
| 4.2 Experimental Setup . . . . .                                                   | 24        |
| 4.3 Evaluation Metrics . . . . .                                                   | 24        |
| 4.4 Dataset . . . . .                                                              | 24        |
| 4.5 Implementation and Results . . . . .                                           | 25        |
| 4.5.1 Backend Implementation . . . . .                                             | 25        |
| 4.5.2 REST API Endpoints . . . . .                                                 | 26        |
| 4.5.3 React Web Frontend Implementation . . . . .                                  | 26        |
| 4.5.4 Flutter Mobile Frontend Implementation . . . . .                             | 28        |
| 4.5.5 Qualitative Results . . . . .                                                | 29        |
| 4.5.6 Quantitative Results . . . . .                                               | 29        |
| 4.5.7 Objective Achievement . . . . .                                              | 31        |
| 4.6 Conclusion . . . . .                                                           | 31        |
| <b>V Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues</b> | <b>32</b> |
| 5.1 Intellectual Property Considerations . . . . .                                 | 32        |

|            |                                                                            |           |
|------------|----------------------------------------------------------------------------|-----------|
| 5.2        | Ethical Considerations . . . . .                                           | 32        |
| 5.3        | Safety Considerations . . . . .                                            | 32        |
| 5.4        | Legal Considerations . . . . .                                             | 32        |
| 5.5        | Impact of the Project on Societal, Health, and Cultural Issues . . . . .   | 33        |
| 5.6        | Impact of Project on the Environment and Sustainability . . . . .          | 33        |
| <b>VI</b>  | <b>Addressing Complex Engineering Problems and Activities</b>              | <b>34</b> |
| 6.1        | Complex Engineering Problems Associated with the Current Project . . . .   | 34        |
| 6.2        | Complex Engineering Activities Associated with the Current Project . . . . | 34        |
| <b>VII</b> | <b>Conclusions</b>                                                         | <b>35</b> |
| 7.1        | Summary . . . . .                                                          | 35        |
| 7.2        | Limitations . . . . .                                                      | 35        |
| 7.3        | Recommendations and Future Works . . . . .                                 | 35        |

## List of Tables

| Table No. | Description                                                     | Page |
|-----------|-----------------------------------------------------------------|------|
| 1.1       | RACI Matrix for Project Work Distribution . . . . .             | 3    |
| 2.1       | Comparison of Related Outpatient Coordination Systems . . . . . | 7    |
| 3.1       | NestJS Backend Module Structure . . . . .                       | 11   |
| 3.2       | Appointment States and Clinical Semantics . . . . .             | 17   |
| 3.3       | Prescription States and Semantics . . . . .                     | 18   |
| 3.4       | Lab Order States and Semantics . . . . .                        | 18   |
| 3.5       | Database Schema — Model Overview . . . . .                      | 21   |
| 4.1       | Principal REST API Endpoint Groups . . . . .                    | 27   |
| 4.2       | React Web Frontend Pages by Role . . . . .                      | 28   |
| 4.3       | Automated Test Suite Results . . . . .                          | 29   |



## List of Figures

| Figure No. | Description                                                              | Page |
|------------|--------------------------------------------------------------------------|------|
| 1.1        | Gantt Chart of Project Timeline . . . . .                                | 4    |
| 3.1        | Overall System Architecture of MedFlow . . . . .                         | 9    |
| 3.2        | Level-1 Data Flow Diagram of MedFlow . . . . .                           | 11   |
| 3.3        | Use Case Diagram of MedFlow — Authentication and Patient Care Access     | 13   |
| 3.4        | Use Case Diagram of MedFlow — Patient and Doctor Workflows . . . . .     | 14   |
| 3.5        | Use Case Diagram of MedFlow — Pharmacy and Diagnostic Workflows . .      | 15   |
| 3.6        | Use Case Diagram of MedFlow — Oversight and System Side Effects . . .    | 16   |
| 3.7        | Appointment Finite-State Machine . . . . .                               | 17   |
| 3.8        | Prescription Finite-State Machine . . . . .                              | 19   |
| 3.9        | Lab Order Finite-State Machine . . . . .                                 | 20   |
| 3.10       | Entity Relationship Diagram of MedFlow . . . . .                         | 22   |
| 4.1        | Appointment Called Notification Sequence . . . . .                       | 27   |
| 4.2        | Doctor Dashboard and Analytics Interface . . . . .                       | 28   |
| 4.3        | Doctor Appointment Details and Prescription Creation Interface . . . . . | 29   |
| 4.4        | Patient Mobile Dashboard Interface . . . . .                             | 30   |
| 4.5        | Distribution of Passing Automated Tests Across Codebases . . . . .       | 31   |

# **CHAPTER I**

## **Introduction**

### **1.1 Introduction**

Healthcare is becoming more and more digitised in terms of appointment management, prescriptions, diagnostic reports, and communication with the stakeholders to minimize wait time and administrative error [1, 2]. Still, in Bangladesh, numerous outpatient services rely on telephone reservations, paper orders and hand coordination between doctors, pharmacies, and diagnostic laboratories.

MedFlow fills this gap as an integrated care coordination system outpatient built. A NestJS backend, a React web application and a Flutter mobile application. It offers role-scoped interfaces between patients, doctors, pharmacies, diagnostic laboratories and administrators via a single API, and via a WebSocket notification layer [3]. Combination of appointment, prescription, is its primary contribution. In a single, coherent workflow, lab-order, document-storage, profile, notification and audit workflows. codebase.

### **1.2 Background / Problem Statement**

Outpatient coordination is often based on traditional outpatient coordination through telephone, paper queues, manual result dispatch, handwritten prescriptions, and handwritten prescriptions, and manual result dispatch [4]. This causes duplicate bookings, lost/unclear prescriptions, late result collection, and poor patient transparency of appointment status.

Current electronic solutions typically address a single aspect of the process. Appointment platforms do not mimic clinical states with confirmation, pharmacy systems do not interface with. Doctors and laboratory systems are not normally connected to appointment records, and laboratory systems. patients. MedFlow thus focuses on the entire process of appointments request and so on. prescription, dispensing, diagnostic reporting and notification.

### 1.3 Objectives

The principal objectives of this project are as follows.

1. To design and build a system that has role based access , that coordinates patients, doctors, pharmacy and diagnostic workflow
2. To implement and test the workflow works from appointment to prescribing to lab results and to check for real time notifications
3. To deliver usable multi-platform interfaces (React web and Flutter mobile) for operational workflow execution, status visibility, and document/report access.

### 1.4 Scope

The project can be divided into three parts : backend , web and mobile. The backend was built on NestJS v11, Prisma ORM v7, PostgreSQL, Socket.IO, Passport/JWT, bcrypt, PDFKit, Cloudinary, class-validator, and dotenv. The web frontend was built using React 19, TypeScript 5.9, React Router, TanStack Query, Axios, Socket.IO client, Recharts, Zod, Vite, Vitest, and Testing Library. For the mobile app we used Flutter/Dart , GoRouter, http, socket\_io\_client, image\_picker, permission\_handler, shared\_preferences, and url\_launcher. Version control was managed with Git and GitHub.

### 1.5 Unfamiliarity of the Problem / Topic / Solution

Appointment systems do exist. There are tools for prescription and lab systems as well. But the thing is they are all each a separated tools of their own. The combined workflow that we created in medflow is something we did not find in the open-source or academic systems we looked into. [4, 6]. It has a seven state appointment FSM , a five-state FSM for prescription. And there are also the lab workflow , role based websocket notifications and a Flutter app for the patient side in our overall system. All are integrated in one system keeping in mind the actual workflow of an actual patient.

Table 1.1: RACI Matrix for Project Work Distribution

| Task                   | Apu | Arfan | Supervisor | Reviewer |
|------------------------|-----|-------|------------|----------|
| Requirements Analysis  | R/A | C     | C          | I        |
| DB Schema & Migrations | C   | R/A   | C          | I        |
| Auth & Notifications   | R/A | C     | I          | I        |
| Appointment Module     | R/A | C     | I          | I        |
| Prescription & Labs    | C   | R/A   | I          | I        |
| Audit & Cloudinary     | C   | R/A   | I          | I        |
| React Web Frontend     | C   | R/A   | I          | C        |
| Flutter Mobile App     | R/A | C     | I          | C        |
| Testing                | R   | R/A   | I          | C        |
| Report Writing         | R/A | R/A   | C          | C        |

## 1.6 Project Planning and Work Distribution

We built the project over ten weeks. Both the members contributed throughout the stack of the system. The responsibilities have been shown in a RACI matrix in (Table 1.1) and the overall timeline of the project has been shown in the Gantt chart (Figure 1.1). Apu Sharma focused on building the backend up from scratch, user level authentication, notification system and the mobile flutter development. MD. Eftakar jaman AArfan designed the schema workflow, the modules, role-specific frontend, the web flow and testing.

*R = Responsible, A = Accountable, C = Consulted, I = Informed*

## 1.7 Applications of the Work

MedFlow can be implemented in government outpatient departments, in private clinics, pharmacies. and diagnostic laboratories. It substitutes physical queues with online status monitoring, enables physicians to schedule appointments and records, allows pharmacies to accept prescriptions. data, and allows diagnostic centres to post reports and inform patients. Its REST Audit trail, API, and typed WebSocket events also form a basis of more comprehensive. outpatient interoperability.

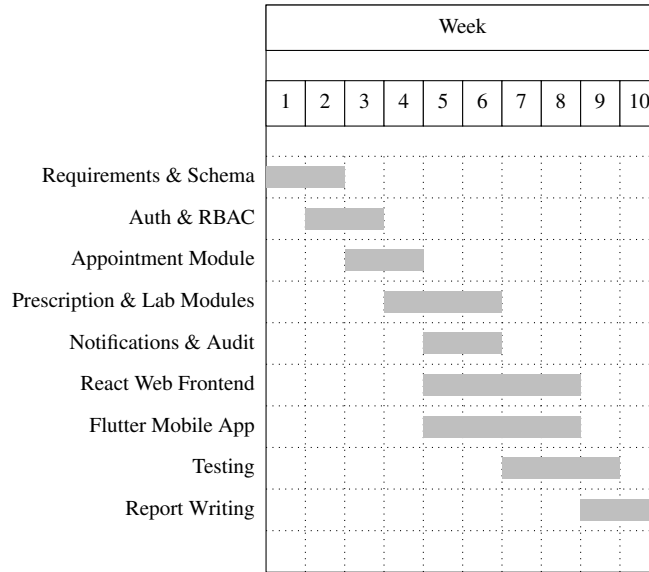


Figure 1.1: Gantt Chart of Project Timeline

## 1.8 Organization of the Project

Chapter II is an overview of the literature on outpatient care coordination, EHR platforms, pharmacy-laboratory integration, and appointment management. The Chapter III describes the system architecture, database schema and design decisions throughout all three codebases. Implementation and results are given in Chapter IV. Chapter V answers societal, ethical, legal and environmental issues. Chapter VI analyses the complicated engineering issues and tasks that are faced. Chapter VII concludes the Summary, limitations and recommendations of future work.

## **CHAPTER II**

### **Related Works**

#### **2.1 Introduction**

Outpatient visits involve patients, doctors, pharmacies, diagnostic laboratories, and administrative staff. Prior work ranges from large EHR platforms to lightweight booking tools and mobile health applications. This chapter positions MedFlow within that landscape as an integrated doctor-patient-pharmacy-laboratory coordination platform.

#### **2.2 Related Works**

##### **2.2.1 Electronic Health Record Systems**

OpenMRS [7] is a widely deployed open-source EHR platform, but its appointment module supports only Scheduled, Completed, and Cancelled states, which is too coarse for queue-aware outpatient coordination. OpenEMR [8] adds billing and prescribing, but has limited mobile support and no real-time WebSocket notification infrastructure. Neither system implements MedFlow’s complete loop: appointment state tracking, prescription forwarding, diagnostic order/result handling, patient mobile notifications, and audit logging from one API.

##### **2.2.2 Commercial Appointment APIs**

Calendly and Acuity Scheduling provide general-purpose booking APIs, but they are limited to host-invitee scheduling and lack pharmacy, diagnostic, prescription, lab-result, and intermediate clinical state concepts [9].

### **2.2.3 Pharmacy and Laboratory Information Systems**

Standalone pharmacy systems focus on inventory, billing, and dispensing, while laboratory systems focus on test registration, sample handling, and report delivery. They are useful inside one organisation, but usually do not receive doctor-generated prescriptions or lab orders from a shared outpatient workflow, nor do they provide unified patient updates.

### **2.2.4 Clinical Workflow Backends**

Reviewed GitHub repositories and academic theses [10] typically implement JWT authentication and basic appointment CRUD. None combine an appointment FSM, prescription and lab-order workflows, cloud document storage, WebSocket push notifications, and both React and Flutter clients from one API.

### **2.2.5 Flutter Healthcare Applications**

Flutter healthcare apps commonly focus on appointment booking or teleconsultation [11]. Reviewed examples either rely on Firebase or focus only on the patient role, without modelling pharmacy and diagnostic stakeholder journeys.

## **2.3 Discussion**

Table 2.1 provides a structured comparison of the reviewed systems against our implementation across the key differentiating dimensions.

The comparison confirms that MedFlow addresses a combination of outpatient coordination requirements not covered by any single reviewed solution, validating the originality of the problem formulation and its implementation.

Table 2.1: Comparison of Related Outpatient Coordination Systems

| <b>System</b>     | <b>Roles</b> | <b>Appt. States</b> | <b>Rx Flow</b> | <b>Real-time</b> | <b>Mobile App</b>            |
|-------------------|--------------|---------------------|----------------|------------------|------------------------------|
| OpenMRS           | 4            | 3                   | No             | No               | No                           |
| OpenEMR           | 3            | 3                   | Partial        | No               | No                           |
| Calendly API      | 2            | 2                   | No             | Webhook          | No                           |
| Reviewed Back-end | 2            | 3                   | No             | No               | No                           |
| Reviewed Flutter  | 1            | 2                   | No             | Firebase         | Patient only                 |
| <b>MedFlow</b>    | <b>5</b>     | <b>7</b>            | <b>5-state</b> | <b>WebSocket</b> | <b>Flutter (Android+iOS)</b> |



## CHAPTER III

### System Analysis and Design

#### 3.1 Introduction

This chapter describes the methodology, architecture, database schema, and design decisions across the three codebases. Development followed an iterative process from requirements and schema design to backend, frontend, testing, and documentation.

#### 3.2 System Analysis and Design Approach

The project followed a five-phase iterative approach.

1. **Phase 1 — Requirements and Schema Design:** Stakeholder analysis, role identification, FSM design, and Prisma schema definition with a migration strategy planned across fifteen incremental migrations.
2. **Phase 2 — Backend Core:** NestJS module scaffolding, authentication, RBAC guards, appointment service and controller.
3. **Phase 3 — Backend Features:** Prescription, lab order, notification, audit, Cloudinary, and profile modules.
4. **Phase 4 — Frontend Development:** React web dashboard and Flutter mobile application built in parallel against the stable API.
5. **Phase 5 — Testing and Documentation:** Backend service/controller tests, web component tests, Flutter unit/widget tests, and report writing.

#### 3.3 Overall Framework and Architecture

Figure 3.1 illustrates the overall system architecture and is rotated to preserve readability.



The system follows a three-tier architecture. The **Presentation Tier** contains the React web SPA and Flutter mobile app, both using REST and Socket.IO. The **Application Tier** is the NestJS server for routing, authentication, business logic, FSM enforcement, PDF generation, and file upload coordination. The **Data Tier** uses PostgreSQL through Prisma and Cloudinary for avatars, prescription PDFs, and lab result files.

### 3.3.1 Data Flow Diagram

Figure 3.2 presents the level-1 data flow diagram of MedFlow, showing the interaction among external actors, major application processes, persistent data stores, and external file/email services.

### 3.3.2 Backend Module Structure

The NestJS application is organised into fourteen feature modules, each encapsulating its own controller, service, and DTOs. Table 3.1 summarises these modules and their responsibilities.

### 3.3.3 Web Frontend Architecture

The React web app uses role-scoped feature folders, guarded nested routes, TanStack Query for server state, and a Socket.IO client for notification events. Recharts powers the Doctor dashboard's weekly appointment and patient-growth charts.

### 3.3.4 Mobile Frontend Architecture

The Flutter app uses a layered architecture: **core/api** provides typed REST and multipart calls, **core/domain** defines Dart models and enums, and **features/patient** separates repositories, mappers, controllers, and pages. GoRouter handles declarative navigation with a persistent `PatientShell`, while `SessionController` drives login redirects. Real-time notifications arrive through `socket_io_client` and update the notifications centre.

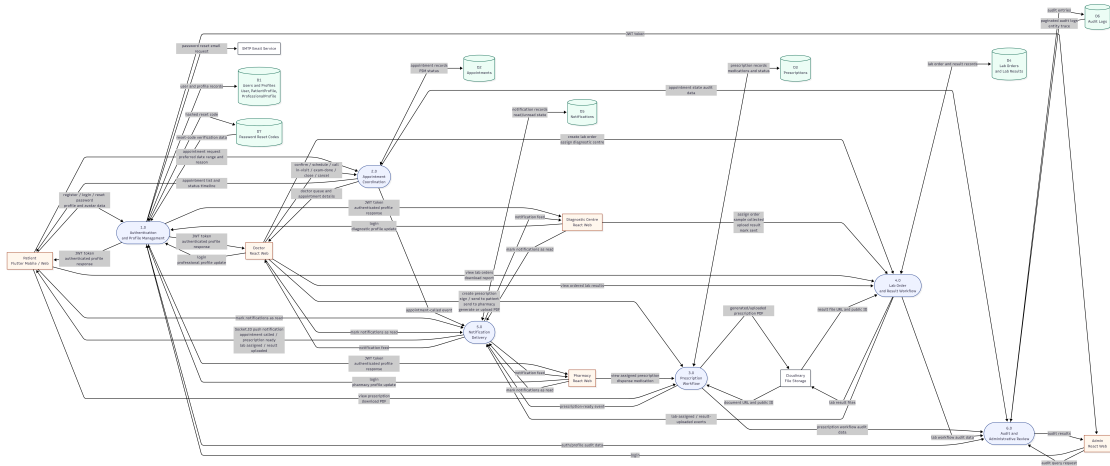


Figure 3.2: Level-1 Data Flow Diagram of MedFlow

Table 3.1: NestJS Backend Module Structure

| Module              | Responsibility                                                                                        |
|---------------------|-------------------------------------------------------------------------------------------------------|
| PrismaModule        | Wraps PrismaClient as a globally injectable singleton service.                                        |
| CloudinaryModule    | Wraps Cloudinary upload API; provides base64 fall-back in development.                                |
| HealthModule        | Exposes a GET /health liveness endpoint.                                                              |
| AuthModule          | Registration, login, forgot-password, reset-password, JWT issuance, JwtAuthGuard, RolesGuard.         |
| UsersModule         | User lookup and admin-facing user management endpoints.                                               |
| PatientsModule      | Patient profile CRUD, doctor search, and avatar upload.                                               |
| DoctorsModule       | Doctor profile CRUD, public profile viewing, and available time slot management.                      |
| PharmaciesModule    | Pharmacy profile CRUD.                                                                                |
| DiagnosticModule    | Diagnostic centre profile CRUD and accreditation management.                                          |
| AppointmentsModule  | Appointment creation, listing, and FSM state transitions; integrates notification and audit services. |
| PrescriptionsModule | Prescription draft creation, PDF generation, status transitions, and pharmacy assignment.             |
| LabsModule          | Lab order creation, assignment, sample collection, and result file upload.                            |
| NotificationsModule | Notification creation, listing, mark-read; Socket.IO WebSocket gateway for real-time push.            |
| AuditModule         | Audit log recording and admin-facing paginated query endpoint.                                        |

## 3.4 Problem Design and Analysis

### 3.4.1 Role Identification

Five actor types were identified from outpatient workflow analysis.

- **Patient** — Registers with a medical profile, browses available doctors, books appointment requests, tracks appointment status, views prescriptions and lab results, and receives real-time push notifications.
- **Doctor** — Manages an appointment queue, advances appointment states, views patient medical histories and clinical records, writes prescriptions, assigns lab orders to diagnostic centres, and accesses an analytics dashboard.
- **Pharmacy** — Receives prescriptions assigned by doctors, views medication details, downloads prescription PDFs, and advances prescription status through to dispensing.
- **Diagnostic** — Receives lab orders assigned by doctors, acknowledges orders, collects samples, uploads result files to Cloudinary, and notifies patients when reports are ready.
- **Admin** — Has system-wide oversight and can access paginated audit logs.

Figures 3.3–3.6 present the MedFlow use case diagram in four readable parts because the full model is too tall for one legible A4 page.

### 3.4.2 Appointment State Machine

The appointment lifecycle was modelled as a seven-state finite-state machine. Table 3.2 defines each state, and Figure 3.7 illustrates the permitted transitions as encoded in the backend source code.

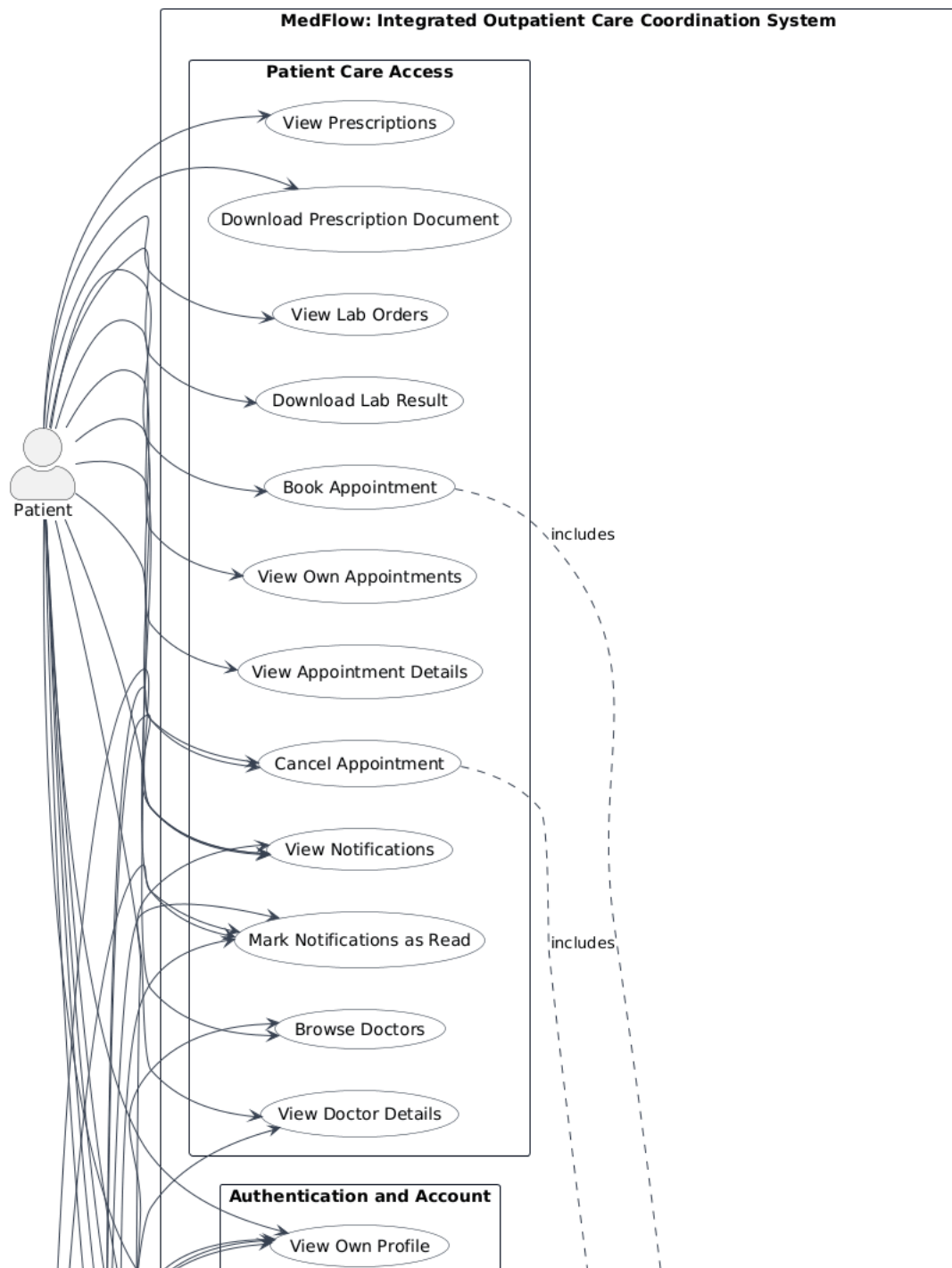


Figure 3.3: Use Case Diagram of MedFlow — Authentication and Patient Care Access

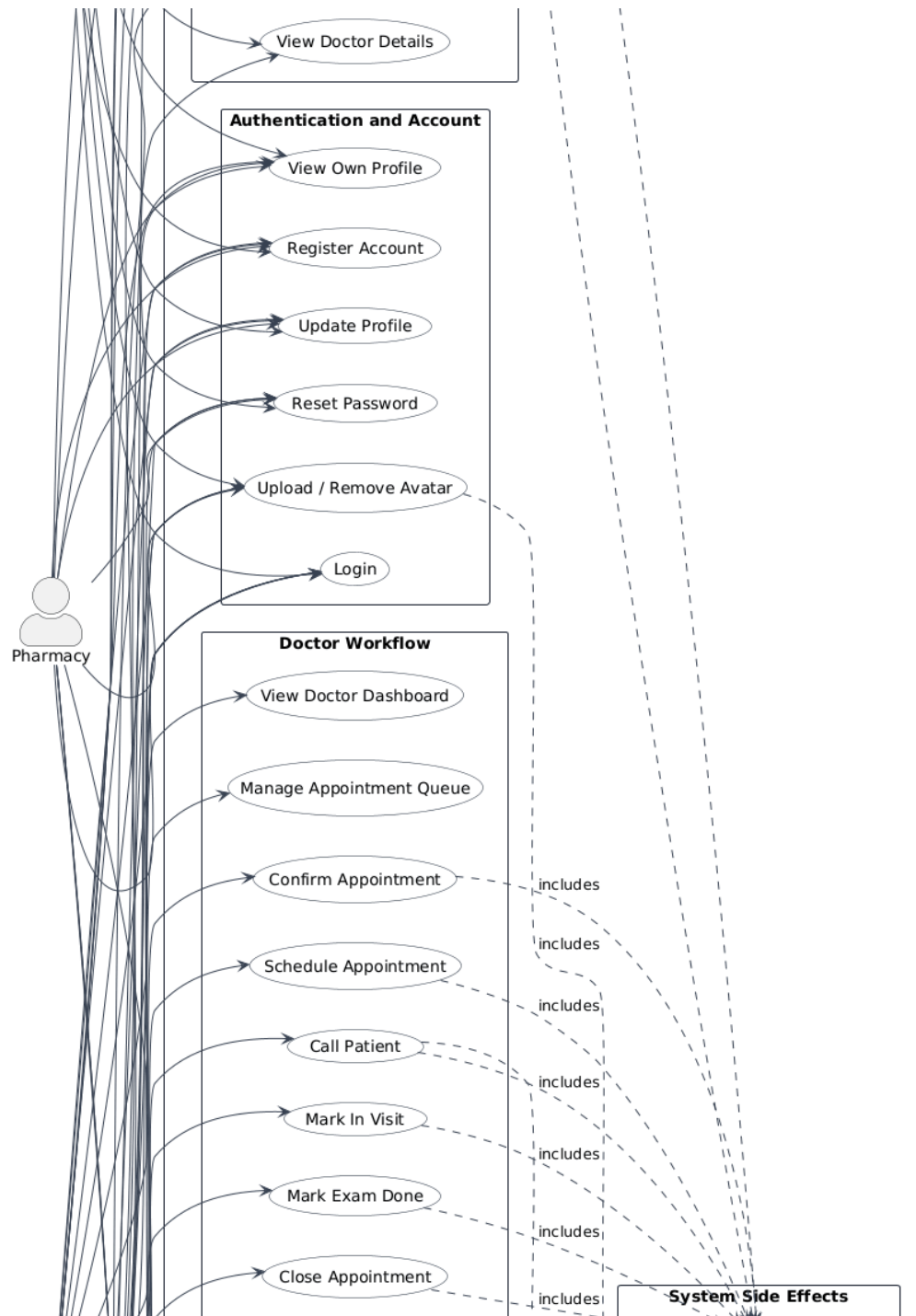


Figure 3.4: Use Case Diagram of MedFlow — Patient and Doctor Workflows

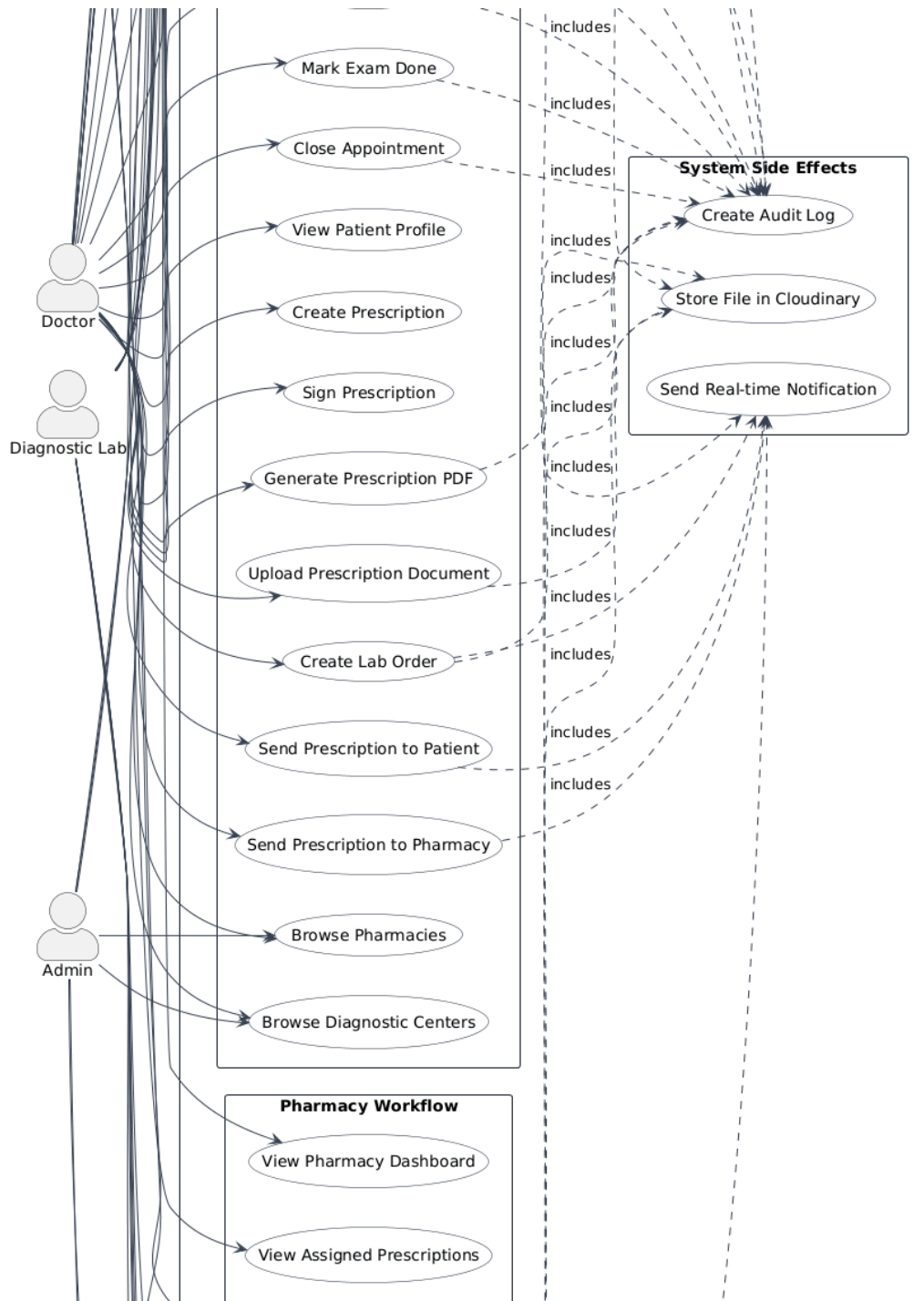


Figure 3.5: Use Case Diagram of MedFlow — Pharmacy and Diagnostic Workflows



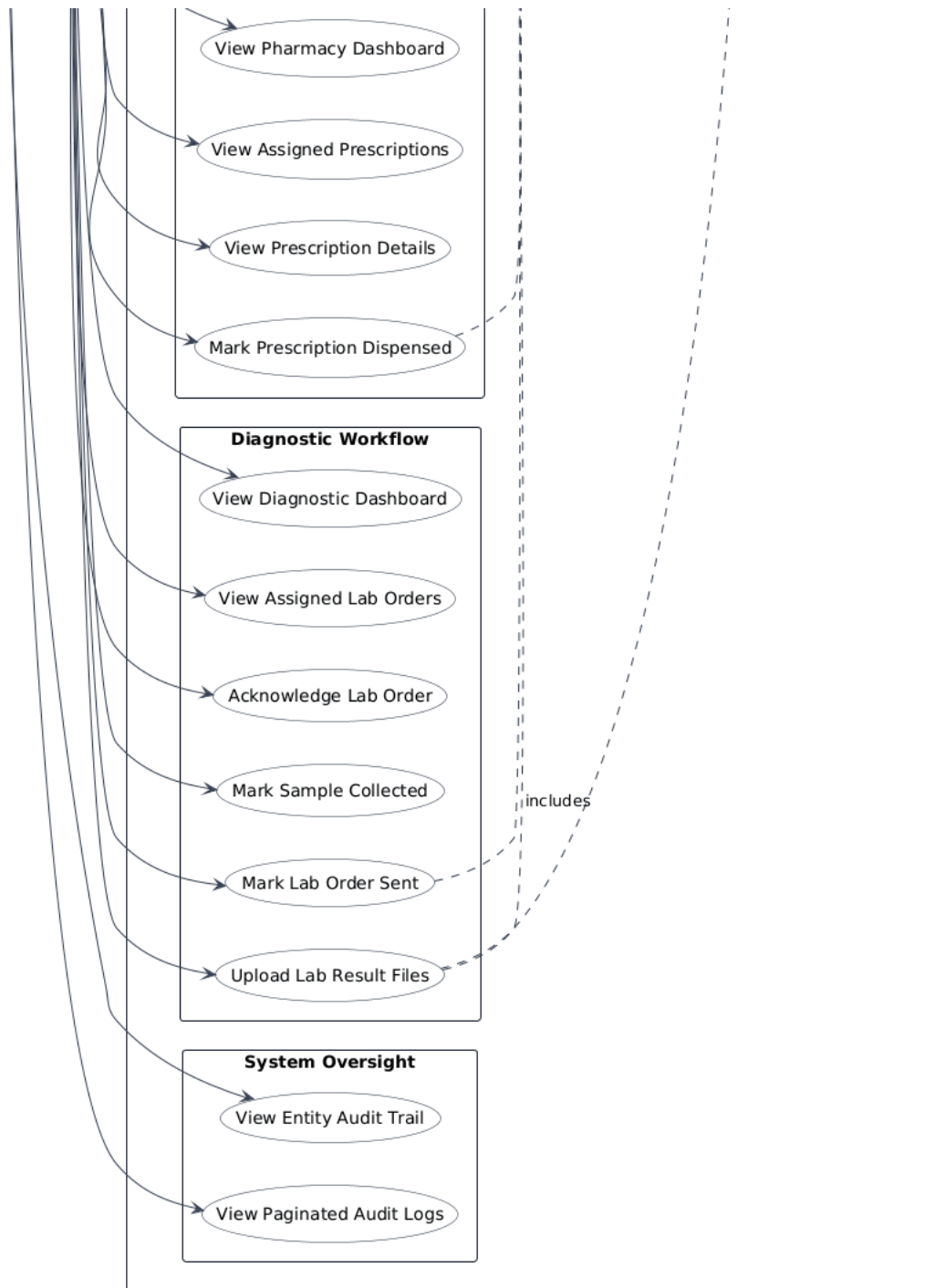


Figure 3.6: Use Case Diagram of MedFlow — Oversight and System Side Effects

Table 3.2: Appointment States and Clinical Semantics

| State     | Semantics                                                                                                                        |
|-----------|----------------------------------------------------------------------------------------------------------------------------------|
| REQUESTED | Patient has submitted a request; awaiting doctor confirmation. A preferred date range and a time note may accompany the request. |
| CONFIRMED | Doctor has accepted and the appointment is scheduled.                                                                            |
| CALLED    | Patient has been called to the waiting area. A push notification is dispatched to the patient at this transition.                |
| IN_VISIT  | Patient is currently in the consultation room.                                                                                   |
| EXAM_DONE | Consultation is complete. Lab orders and prescriptions may now be created.                                                       |
| CLOSED    | All associated steps are resolved. Appointment is finalised.                                                                     |
| CANCELLED | Appointment was cancelled by patient, doctor, or admin.                                                                          |

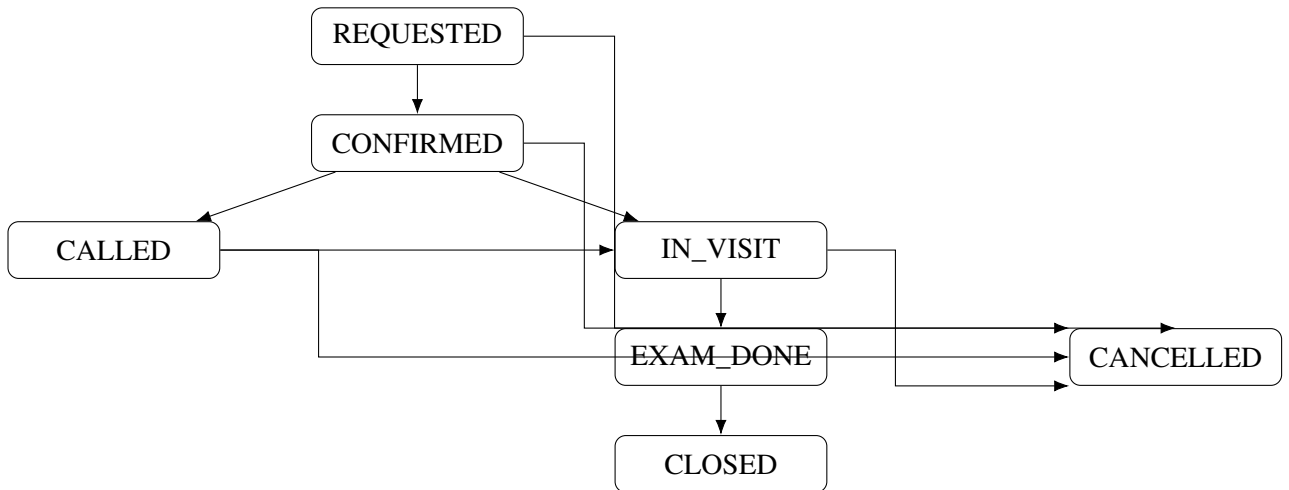


Figure 3.7: Appointment Finite-State Machine

Table 3.3: Prescription States and Semantics

| State            | Semantics                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------|
| DRAFT            | Doctor has created the prescription but not yet finalised it.                                      |
| SIGNED           | Doctor has signed the prescription; the generated PDF document can then be uploaded to Cloudinary. |
| SENT_TO_PATIENT  | Prescription is visible to the patient; a push notification is dispatched.                         |
| SENT_TO_PHARMACY | Prescription has been forwarded to the assigned pharmacy.                                          |
| DISPENSED        | Pharmacy has dispensed the medication; the workflow is complete.                                   |

Table 3.4: Lab Order States and Semantics

| State            | Semantics                                                                                              |
|------------------|--------------------------------------------------------------------------------------------------------|
| CREATED          | Doctor has created the order and assigned a diagnostic centre.                                         |
| ASSIGNED         | Diagnostic centre has acknowledged the order; push notification dispatched to patient.                 |
| SAMPLE_COLLECTED | Diagnostic centre has collected the biological sample.                                                 |
| SENT             | Result file uploaded to Cloudinary; push notification dispatched; patient may now download the report. |

### 3.4.3 Prescription State Machine

Prescriptions follow an independent five-state lifecycle, also enforced by a transition map in the prescriptions service. Table 3.3 defines each state.

### 3.4.4 Lab Order State Machine

Lab orders follow a structured four-state lifecycle as defined in Table 3.4.

## 3.5 Database Schema Design

The Prisma schema defines ten models. Table 3.5 provides a high-level overview of each model, its key fields, and its relationships.

Figure 3.10 shows the entity relationships generated from the actual Prisma schema.

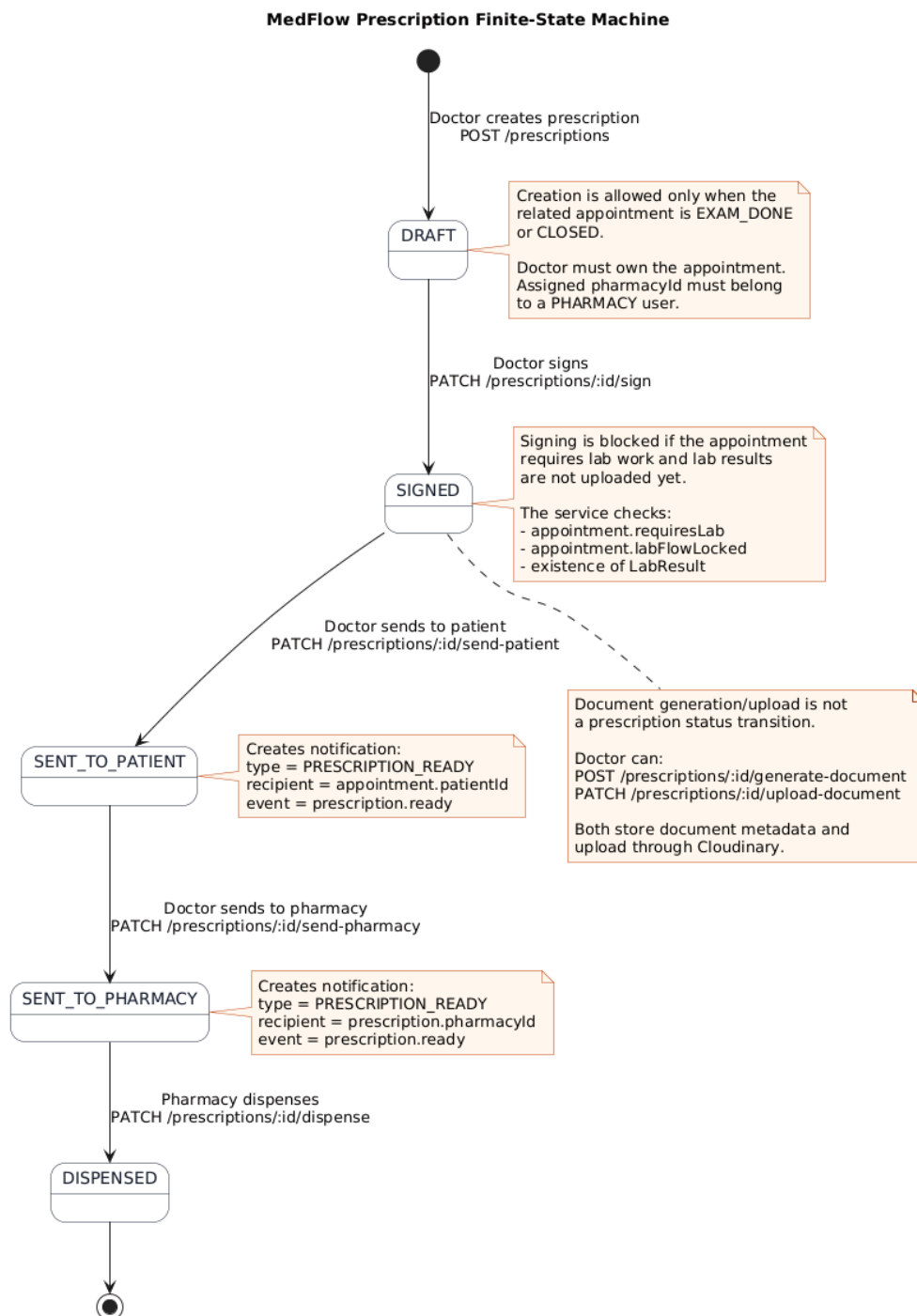


Figure 3.8: Prescription Finite-State Machine

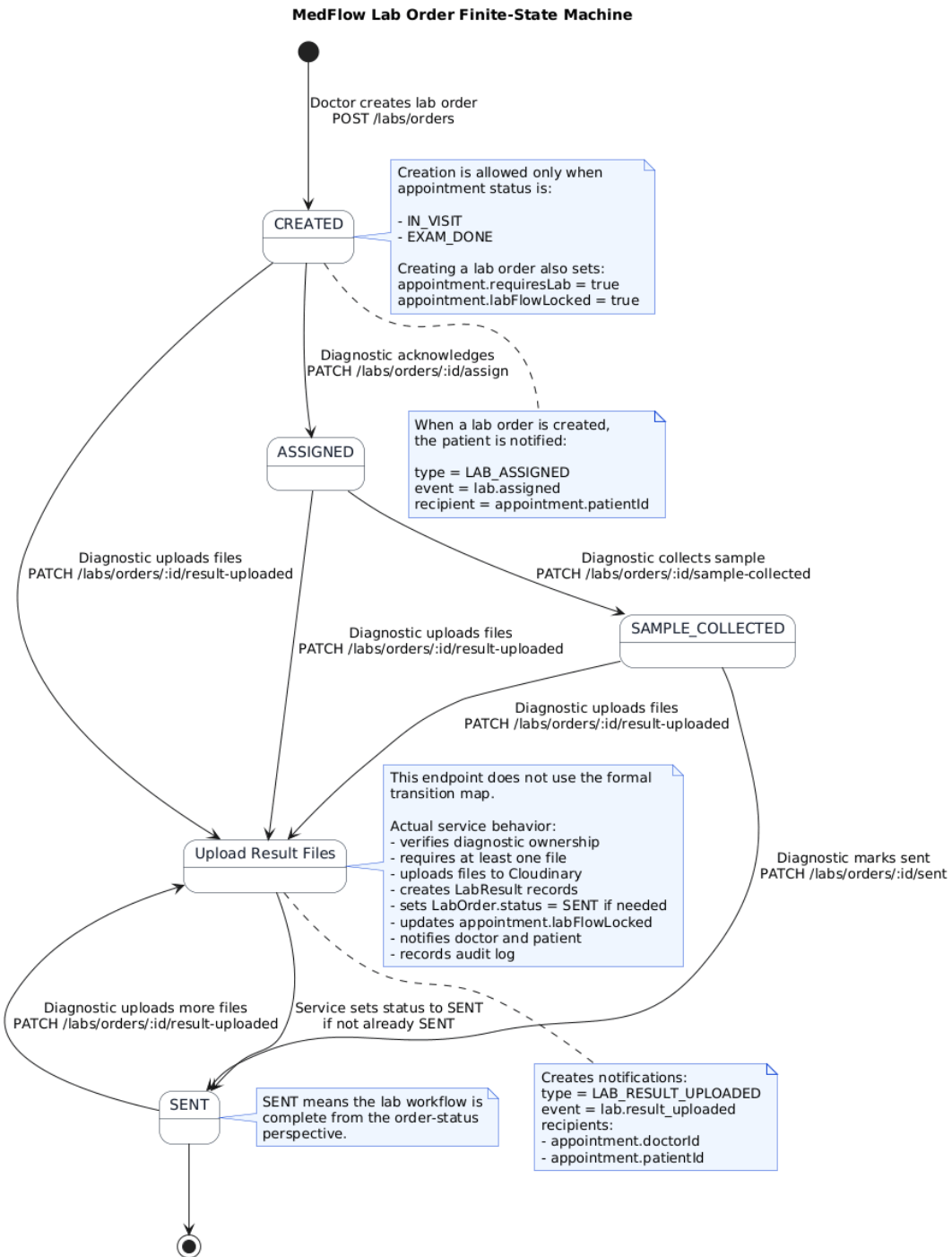


Figure 3.9: Lab Order Finite-State Machine

Table 3.5: Database Schema — Model Overview

| Model               | Key Fields and Relationships                                                                                                                                                                                                                                                                    |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User                | UUID PK, fullName, avatarUrl, email (unique), passwordHash, role (enum), phone, address; relates to PatientProfile, ProfessionalProfile, Appointments (as patient and doctor), LabOrders (as diagnostic), Prescriptions (as doctor and pharmacy), Notifications, AuditLogs, PasswordResetCodes. |
| PatientProfile      | One-to-one with User; stores dateOfBirth, gender, allergies, chronicConditions, currentMedications, and emergency contact fields.                                                                                                                                                               |
| ProfessionalProfile | One-to-one with User; stores licenseNumber, specialization, about, clinicName, clinicAddress, availableTimeSlots (JSON), degrees (JSON), certifications (JSON), yearsOfExperience, availableTests (JSON); fields used selectively based on role.                                                |
| Appointment         | UUID PK; patientId and doctorId FKs to User; status (AppointmentStatus enum); scheduledAt, reason, preferredDateFrom, preferredDateTo, preferredTimeNote, requiresLab, labFlowLocked; relates to LabOrders and Prescriptions.                                                                   |
| LabOrder            | UUID PK; appointmentId FK; diagnosticId FK to User; status (LabOrderStatus enum); tests (JSON array); relates to LabResults.                                                                                                                                                                    |
| LabResult           | UUID PK; labOrderId FK; fileUrl, filePublicId, fileMimeType, fileSizeBytes; uploadedAt timestamp.                                                                                                                                                                                               |
| Prescription        | UUID PK; appointmentId FK; doctorId and pharmacyId FKs to User; notes, diagnosis, instructions, medications (JSON); status (PrescriptionStatus enum); documentUrl, documentPublicId, documentVersion.                                                                                           |
| Notification        | UUID PK; userId FK; type (NotificationType enum: APPOINTMENT_CALLED, LAB_ASSIGNED, LAB_RESULT_UPLOADED, PRESCRIPTION_READY); message; read boolean.                                                                                                                                             |
| AuditLog            | UUID PK; actorUserId FK; action, entityType, entityId strings; metadata (JSON); createdAt.                                                                                                                                                                                                      |
| PasswordResetCode   | UUID PK; userId FK; email; codeHash (SHA-256); expiresAt; consumedAt; attemptCount; rate-limited to 5 attempts.                                                                                                                                                                                 |

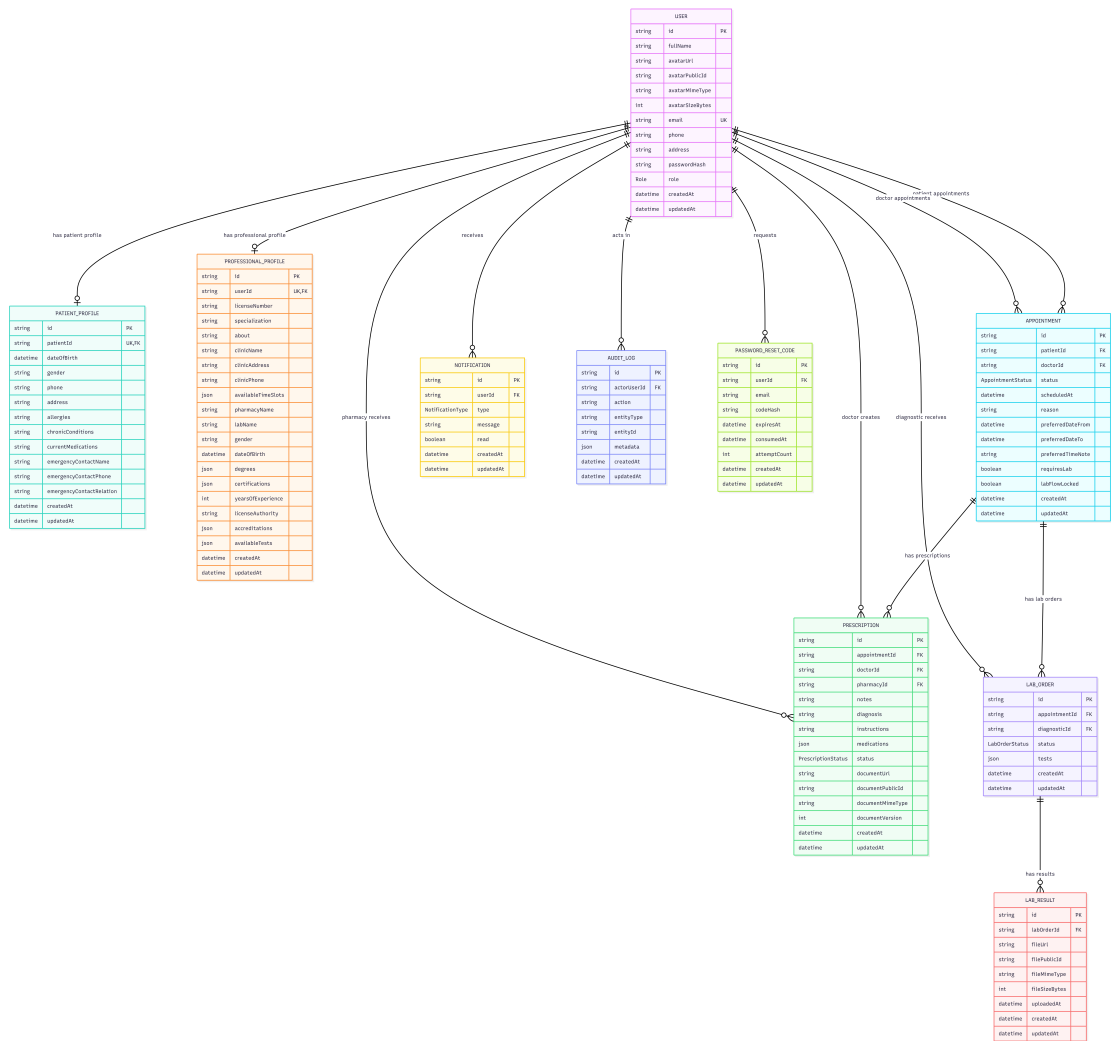


Figure 3.10: Entity Relationship Diagram of MedFlow

### **3.6 Conclusion**

The system analysis and design process kept decisions grounded in outpatient workflows. Prisma acts as the data-layer source of truth, NestJS modules separate backend concerns, and the web and mobile clients share the same API contract through TypeScript and Dart models.



## CHAPTER IV

### Implementation, Results and Discussions

#### 4.1 Introduction

This chapter presents implementation details, API design, frontend structure, and test results across the three application components.

#### 4.2 Experimental Setup

All components were developed and tested on Ubuntu 22.04 LTS using Node.js v20, npm v10, NestJS CLI v11, Prisma CLI v7, PostgreSQL in Docker, TypeScript v5.9, Flutter/Dart 3.11, Vite v7, Vitest with Testing Library, and Flutter Test.

#### 4.3 Evaluation Metrics

Evaluation covered backend service/controller correctness, role enforcement, FSM integrity, notification dispatch to authenticated rooms, schema-level data integrity, and frontend behaviour for role-gated pages, loading states, and error states.

#### 4.4 Dataset

Synthetic test data was created with `backend/prisma/seed.ts`. It includes users for all five roles, patient and professional profiles, appointments, prescriptions, lab orders, lab results, and notifications, providing a realistic baseline for manual and automated testing.

## 4.5 Implementation and Results

### 4.5.1 Backend Implementation

#### Authentication and Registration

Registration accepts role-specific profile data in one request. Patients submit medical profile fields, while doctors, pharmacies, and diagnostic users submit professional profile fields. Passwords are hashed with bcrypt, and login issues a JWT containing the user UUID, email, and role. The password reset flow stores a SHA-256 hash of a six-digit code with expiry, attempt limiting, and consumed-state protection.

#### Appointment Service

The appointment service encodes permitted transitions in a lookup map (Listing IV.1). Trigger transitions, such as CALLED, persist the state change and dispatch a Socket.IO notification to the patient's authenticated room.

```
1 const TRANSITIONS: Record<AppointmentStatus, AppointmentStatus[]> = {  
2   REQUESTED: [CONFIRMED, CANCELLED],  
3   CONFIRMED: [CALLED, IN_VISIT, CANCELLED],  
4   CALLED:    [IN_VISIT, CANCELLED],  
5   IN_VISIT:  [EXAM_DONE, CANCELLED],  
6   EXAM_DONE: [CLOSED],  
7   CLOSED:    [],  
8   CANCELLED: [],  
9 };;
```

Listing IV.1: Appointment FSM Transition Map

#### Prescription Service

Prescriptions can be created only after the appointment reaches EXAM\_DONE or CLOSED. Doctors sign prescriptions, generate or upload PDF documents through Cloudinary, and advance the prescription FSM. When a prescription reaches SENT\_TO\_PATIENT, the

system dispatches a `PRESCRIPTION_READY` notification.

### Lab Order and Result Service

Doctors create lab orders with a diagnostic centre and requested tests. `ASSIGNED` and `SENT` transitions notify the patient; result upload also stores the file in Cloudinary and creates a `LabResult` record.

### WebSocket Notification Gateway

The `NotificationsGateway` authenticates each `Socket.IO` connection with JWT and joins it to `user:{userId}`. Services call `emitToUser(userId, event, payload)` to broadcast only to that room; invalid tokens are disconnected immediately.

Figure 4.1 shows the implemented sequence for advancing an appointment to `CALLED`, including authorization, transition validation, notification persistence, `Socket.IO` emission, and audit logging.

### Cloudinary Service

The `CloudinaryService` signs upload requests with HMAC-SHA256 and returns the URL, public ID, version, MIME type, and byte size. Without Cloudinary credentials, it returns a base64 data URI so development workflows still function.

## 4.5.2 REST API Endpoints

Table 4.1 summarises the main API groups. Controllers expose specific transition endpoints instead of a generic status endpoint, allowing invalid workflow changes to be rejected clearly.

## 4.5.3 React Web Frontend Implementation

The web application provides dedicated dashboards for Patient, Doctor, Pharmacy, and Diagnostic roles. After login, `RoleHomeRedirect` sends each user to the correct

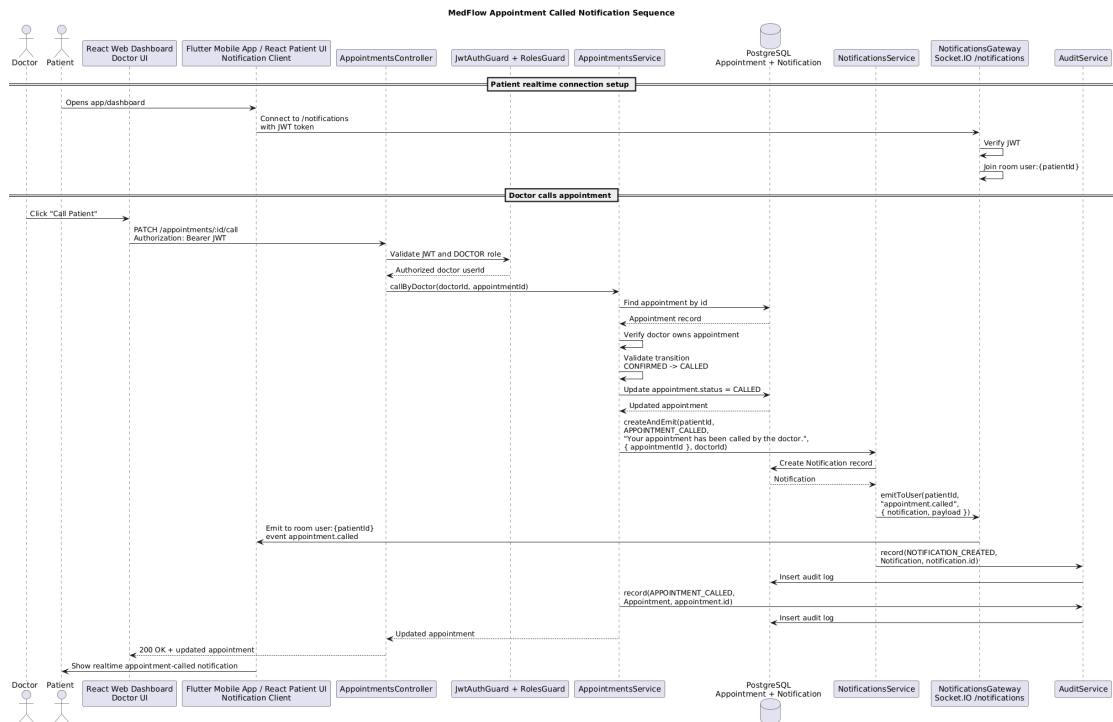


Figure 4.1: Appointment Called Notification Sequence

Table 4.1: Principal REST API Endpoint Groups

| Area           | Representative Paths                                                                                                    | Roles                                    | Purpose                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------|-------------------------------------------------------------|
| Authentication | /auth/register, /auth/login, /auth/forgot-password, /auth/reset-password                                                | Public                                   | Registration, JWT login, and password reset.                |
| Appointments   | /appointments, /appointments/me, /appointments/:id/confirm, /schedule, /call, /in-visit, /exam-done, /close, /cancel    | Patient, Doctor                          | Booking and appointment FSM transitions.                    |
| Prescriptions  | /prescriptions, /prescriptions/:id/sign, /send-patient, /send-pharmacy, /dispense, /upload-document, /generate-document | Doctor, Pharmacy                         | Prescription life-cycle, PDF generation, and dispensing.    |
| Lab Orders     | /labs/orders, /labs/orders/me, /labs/orders/:id/assign, /sample-collected, /result-uploaded, /sent, /result             | Doctor, Diagnostic, Patient              | Lab order creation, diagnostic workflow, and result access. |
| Notifications  | /notifications/me, /notifications/me/read                                                                               | All authenticated users                  | Notification listing and read-state updates.                |
| Profiles       | /users/doctors, /users/me/avatar, /doctors/me/profile                                                                   | Patient, Doctor, all authenticated users | Doctor browsing and profile/avatar management.              |
| Audit          | /audit, /audit/me, /audit/entity/:entityType/:entityId                                                                  | Admin, authenticated users               | Paginated audit review and entity-specific traces.          |

Table 4.2: React Web Frontend Pages by Role

| Role       | Pages Implemented                                                                                                                                                                                                             |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Patient    | Appointments list, Appointment details (status timeline, prescription cards, lab order cards), Doctor details, Notifications, Medical records, Profile with avatar upload.                                                    |
| Doctor     | Home dashboard (weekly appointment bar chart, patient growth line chart, recent activity feed), Appointments list and detail, Patient list and individual patient profile, Prescriptions, Lab orders, Notifications, Profile. |
| Pharmacy   | Home, Prescriptions list, Prescription detail (medication table, download PDF), Notifications, Profile.                                                                                                                       |
| Diagnostic | Home, Lab orders list, Lab order detail (test list, upload result file), Notifications, Profile.                                                                                                                              |

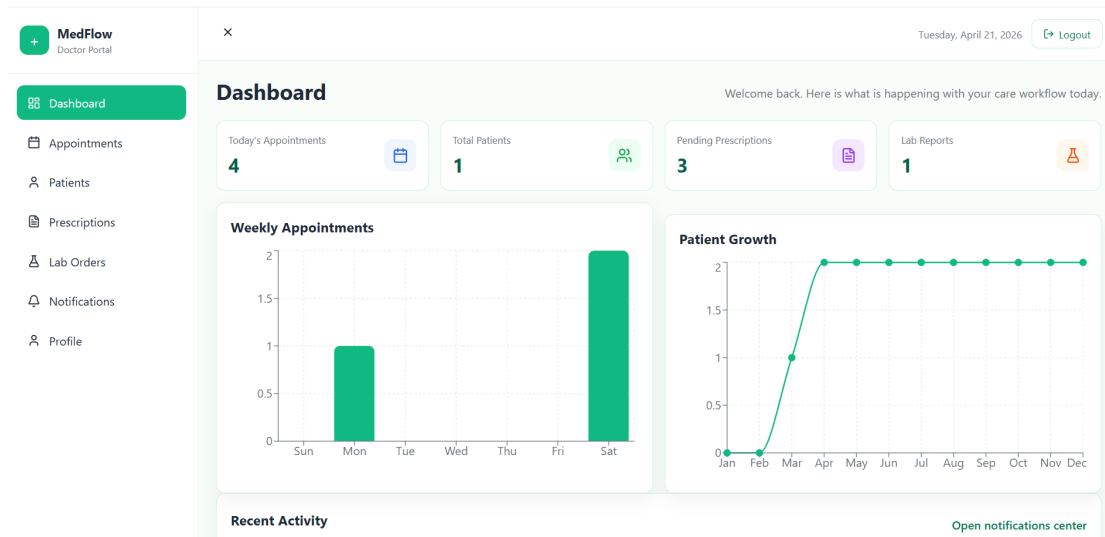


Figure 4.2: Doctor Dashboard and Analytics Interface

home route. Table 4.2 summarises the implemented pages.

The Doctor home page fetches appointments, lab orders, prescriptions, and notifications with TanStack Query, then renders weekly appointment and patient-growth analytics using Recharts.

#### 4.5.4 Flutter Mobile Frontend Implementation

The Flutter application implements the patient journey: authentication, doctor browsing, appointment booking, appointment details, prescription and report downloads, medical records, notifications, and profile/avatar management. GoRouter and PatientShell provide guarded navigation and a persistent bottom bar, while SessionController stores

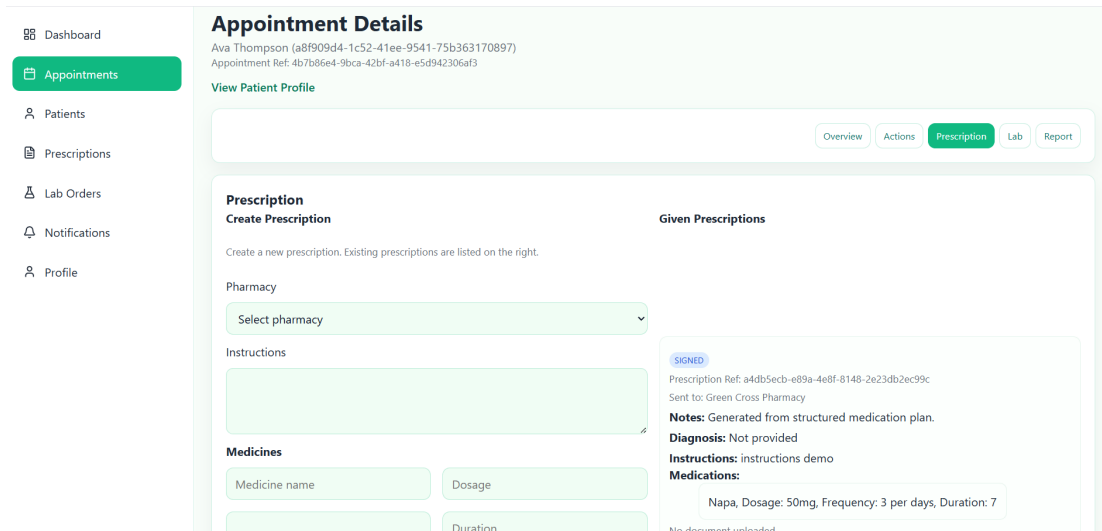


Figure 4.3: Doctor Appointment Details and Prescription Creation Interface

Table 4.3: Automated Test Suite Results

| Test Category                      | Total      | Passed     | Failed   |
|------------------------------------|------------|------------|----------|
| Backend — Jest unit tests          | 102        | 102        | 0        |
| Web — Vitest unit/component tests  | 79         | 79         | 0        |
| Mobile — Flutter unit/widget tests | 75         | 75         | 0        |
| <b>Total</b>                       | <b>256</b> | <b>256</b> | <b>0</b> |

the JWT and triggers redirects. Real-time notifications update the unread count through `NotificationsRealtimeService` and `NotificationsCenterController`.

#### 4.5.5 Qualitative Results

Manual testing used the React web app, Flutter Android emulator, and Thunder Client. Role redirects, FSM transitions, notifications, uploads, downloads, and doctor analytics worked as expected.

#### 4.5.6 Quantitative Results

Table 4.3 summarises the results of the automated test suites across all three code-bases. The backend result is the default `npm test` suite; the separate backend `test:e2e` command is not included in this count.

Figure 4.5 visualises the same passing-test distribution; the table remains the source

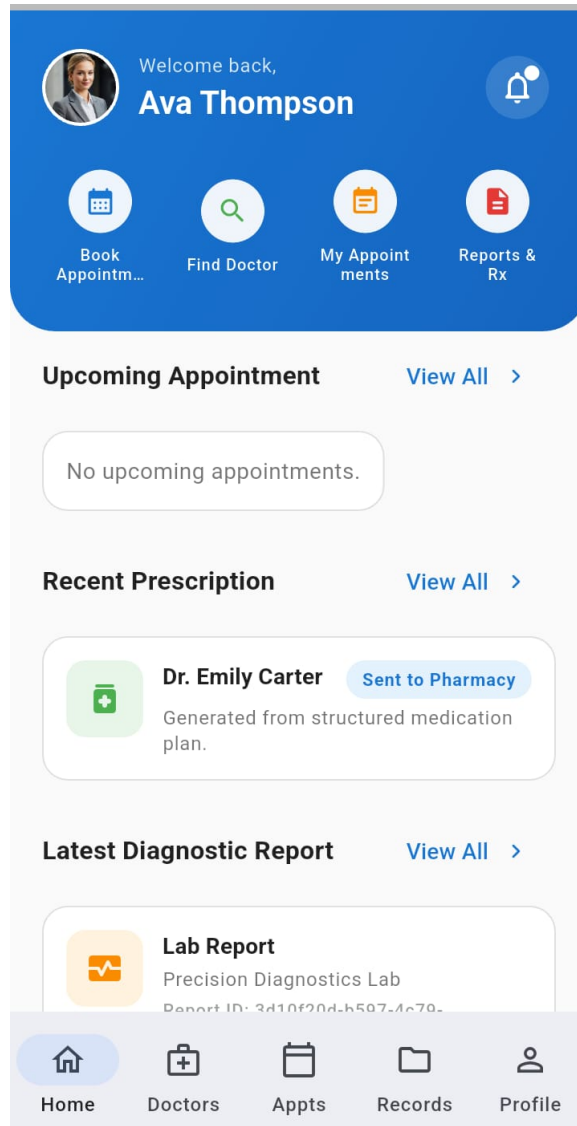


Figure 4.4: Patient Mobile Dashboard Interface

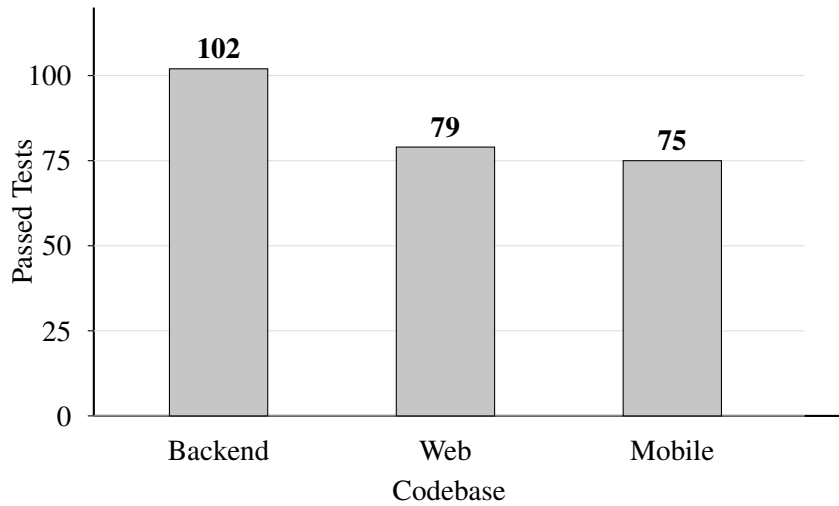


Figure 4.5: Distribution of Passing Automated Tests Across Codebases

for exact totals.

#### 4.5.7 Objective Achievement

All eight objectives from Chapter I were met: five-role authentication, appointment, prescription and lab-order FSMs, PDF and Cloudinary workflows, authenticated WebSocket notifications, role-specific React dashboards, the Flutter patient app, and audit logging for significant state changes.

## 4.6 Conclusion

The implementation demonstrates that a NestJS and Prisma backend, serving both a React SPA and a Flutter mobile client through a common REST and WebSocket API, can effectively model and enforce the complex role and lifecycle requirements of a real-world outpatient care coordination scenario.



## **CHAPTER V**

# **Societal, Health, Environment, Safety, Ethical, Legal and Cultural Issues**

### **5.1 Intellectual Property Considerations**

The project uses permissive open-source tools including NestJS, Prisma, React, Flutter, PostgreSQL, PDFKit, Socket.IO, and the Cloudinary SDK. No proprietary third-party code or real patient data was incorporated; the project codebase is original author work.

### **5.2 Ethical Considerations**

The system handles sensitive health information such as medical histories, diagnoses, prescriptions, and diagnostic results. It applies data minimisation, bcrypt password hashing, role-based access, audit logging, and secure password reset codes with hashing, expiry, attempt limits, and consumed-state protection.

### **5.3 Safety Considerations**

Input validation is enforced through DTO decorators, FSM validators prevent clinically nonsensical states, JWT expiry limits token exposure, invalid Socket.IO clients are disconnected, Prisma parameterises database queries, and Cloudinary uploads are HMAC-SHA256 signed.

### **5.4 Legal Considerations**

Production deployment in Bangladesh would require compliance with the Digital Security Act 2018. Deployments serving European Union residents would also require GDPR-aligned erasure, portability, retention, and privacy processes. Full compliance would

additionally need encryption at rest, a privacy notice, retention/deletion policy, and data protection assessment.

## **5.5 Impact of the Project on Societal, Health, and Cultural Issues**

MedFlow can reduce crowded waiting-room queues through real-time appointment updates, lower prescription loss or misinterpretation through digital delivery, and reduce travel for lab report collection. It also fits the Bangladeshi norm of family involvement by making appointment and result information accessible on a personal device.

## **5.6 Impact of Project on the Environment and Sustainability**

Digitising appointments, prescriptions, and diagnostic reports reduces paper use and unnecessary travel. Cloud hosting can also reduce idle infrastructure compared with dedicated on-premise servers, lowering the environmental footprint of healthcare administration.

## CHAPTER VI

### Addressing Complex Engineering Problems and Activities

#### 6.1 Complex Engineering Problems Associated with the Current Project

Several complex problems were addressed. First, the appointment, prescription, and lab order FSMs each required transition validation plus side effects such as notification, PDF generation, Cloudinary upload, and audit logging without leaving inconsistent state. Second, the WebSocket gateway needed secure reconnection behaviour, solved through JWT-authenticated per-user rooms. Third, the Flutter app needed consistent authentication state across navigation and real-time events, handled through a `SessionController` observed by `GoRouter`. Fourth, Prisma required explicit named relations where `Appointment` references `User` as both patient and doctor.

#### 6.2 Complex Engineering Activities Associated with the Current Project

The work included database evolution through fifteen Prisma migrations, a prescription PDF pipeline spanning JSON medication data, PDFKit buffers, Cloudinary uploads, and stored document URLs, and a layered Flutter architecture with models, repositories, mappers, and controllers. Testing also required backend Jest mocks, React/Vitest role-flow tests, and Flutter unit/widget tests.

## **CHAPTER VII**

### **Conclusions**

#### **7.1 Summary**

This project designed, implemented, and evaluated MedFlow: An Integrated Out-patient Care Coordination System. The NestJS/Prisma/PostgreSQL backend implements ten models, five roles, appointment/prescription/lab FSMs, WebSocket notifications, PDF generation, Cloudinary storage, audit logging, and password reset. The React web frontend supports role-specific dashboards and workflows, while the Flutter mobile frontend provides the patient experience for doctor browsing, booking, notifications, prescriptions, and lab reports. The verified automated suites passed 256 tests across backend, web, and mobile.

#### **7.2 Limitations**

The current implementation has five main limitations: the mobile app covers only patients; load testing and Redis-backed Socket.IO scaling have not been performed; no formal OWASP Top 10 security audit has been completed; email delivery is still a prototype stub; and appointment requests do not yet enforce doctor availability slots.

#### **7.3 Recommendations and Future Works**

Future work should extend mobile support to doctor, pharmacy, and diagnostic roles, integrate a production email provider, add a Redis adapter for Socket.IO scaling, and validate appointment requests against doctor availability. Additional extensions include WebRTC teleconsultation, an Admin audit dashboard, national patient identifier integration, ML-assisted no-show/specialist/prescription analysis, and formal security and privacy audits before real patient deployment.

## References

- [1] Bates, D. W. and Gawande, A. A., 2003, “Improving Safety with Information Technology,” *New England Journal of Medicine*, Vol. 348, No. 25, pp. 2526–2534.
- [2] Shortliffe, E. H. and Cimino, J. J. (Eds.), 2014, *Biomedical Informatics: Computer Applications in Health Care and Biomedicine*, 4th ed., Springer, New York.
- [3] Richardson, L. and Ruby, S., 2007, *RESTful Web Services*, O’Reilly Media, Sebastopol, CA.
- [4] Hossain, M. S. and Ahmed, F., 2019, “Challenges of Healthcare Information Systems in Bangladesh,” *International Journal of Healthcare Information Systems and Informatics*, Vol. 14, No. 2, pp. 1–15.
- [5] Kamrul, I. and Rahman, M., 2021, “A Review of Appointment Scheduling Systems for Outpatient Clinics in South Asia,” *Journal of Health Informatics in Developing Countries*, Vol. 15, No. 1, pp. 45–60.
- [6] Witten, I. H. and Frank, E., 2011, *Data Mining: Practical Machine Learning Tools and Techniques*, 3rd ed., Morgan Kaufmann, Burlington, MA.
- [7] Mamlin, B. W. *et al.*, 2006, “Cooking Up an Open-Source EMR for Developing Countries: OpenMRS — A Recipe for Successful Collaboration,” *AMIA Annual Symposium Proceedings*, pp. 529–533.
- [8] Bowen, W., 2007, “OpenEMR: An Open-Source Electronic Medical Record Application,” *Linux Journal*, Vol. 2007, No. 158, pp. 2.
- [9] Fielding, R. T., 2000, *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. dissertation, University of California, Irvine.
- [10] Kaminsky, D. and Shah, R., 2022, “NestJS in Enterprise Healthcare Backends: A Systematic Review,” *IEEE International Conference on Software Engineering*, pp. 112–119.
- [11] Bui, T. A. and Nguyen, H. T., 2021, “Flutter-Based Telemedicine and Appointment Applications: Design Patterns and Challenges,” *International Journal of Mobile Computing and Multimedia Communications*, Vol. 12, No. 3, pp. 18–34.

- [12] Prisma, 2023, “Prisma ORM Documentation,” Available online:  
<https://www.prisma.io/docs>, Accessed: December 2023.
- [13] NestJS, 2023, “NestJS Documentation,” Available online:  
<https://docs.nestjs.com>, Accessed: December 2023.
- [14] Flutter, 2023, “Flutter Documentation,” Available online:  
<https://docs.flutter.dev>, Accessed: December 2023.